Strings in Java

Junior Level: String Algorithms, Regular Expressions and Unicode

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-13

Outline

1. The String Matching Problem
2. Knuth–Morris–Pratt
3. Rabin–Karp
4. Edit Distance
5. Regular Expressions
6. Unicode and Encodings
7. Summary

The String Matching Problem

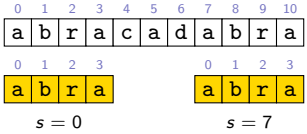
The string matching problem

Given a text $T[0..n-1]$ and a pattern $P[0..m-1]$ over an alphabet Σ.
Find every **shift** s with $0 \leq s \leq n-m$ such that

$$T[s+j] = P[j] \quad \text{for all } 0 \leq j < m$$

Used in editors (*find*), `grep`, search engines, DNA analysis, intrusion detection, plagiarism checks. In Java: `indexOf`, `contains`, `split`, `replace`.

$T =$ `abracadabra`, $P =$ `abra`: shifts 0 and 7.



Question

How many character comparisons does it take? Naively up to $(n-m+1) \cdot m$. Can we do $O(n+m)$?

The naive algorithm

Algorithm 1: NaiveMatcher(T, P)

```
1 n ← |T|,    m ← |P|
2 for s ← 0 to n − m do
3     j ← 0
4     while j < m and T[s + j] = P[j] do
5         j ← j + 1
6     if j = m then
7         report a match at shift s
```

- Try every shift; compare left to right until a mismatch
- **Worst case** $\Theta((n-m+1)m)$: $T = \mathtt{a}^{n-1}\mathtt{b}$, $P = \mathtt{a}^{m-1}\mathtt{b}$
- **Typical text**: mismatches come early, so it is fast in practice
- Java's `String.indexOf` is essentially this (with tuned, vectorized loops)
- The weakness: after a mismatch it **forgets** what it has already seen

The naive algorithm in Java

codeoutput/NaiveSearch.java — Java

```java
import java.util.ArrayList;
import java.util.List;
public class NaiveSearch {
    static long comparisons = 0;
    static List<Integer> search(String t, String p) {
        List<Integer> hits = new ArrayList<>();
        int n = t.length(), m = p.length();
        for (int s = 0; s <= n - m; s++) {                  // every shift
            int j = 0;
            while (j < m) {
                comparisons++;
                if (t.charAt(s + j) != p.charAt(j)) break;  // mismatch
                j++;
            }
            if (j == m) hits.add(s);
        }
        return hits;
    }
    public static void main(String[] args) {
        System.out.println(search("abracadabra", "abra") + " " + comparisons);
        comparisons = 0;
        String t = "a".repeat(1000) + "b";                  // worst case
        String p = "a".repeat(10) + "b";
        System.out.println(search(t, p) + " " + comparisons + " comparisons");
        System.out.println("n * m = " + t.length() * p.length());
    }
}
```

On the worst case ($n = 1001$, $m = 11$) it makes 10 901 comparisons, close to $n \cdot m = 11\,011$.

The naive algorithm in Java

codeoutput/NaiveSearch.java — Java

```java
import java.util.ArrayList;
import java.util.List;
public class NaiveSearch {
    static long comparisons = 0;
    static List<Integer> search(String t, String p) {
        List<Integer> hits = new ArrayList<>();
        int n = t.length(), m = p.length();
        for (int s = 0; s <= n - m; s++) {                  // every shift
            int j = 0;
            while (j < m) {
                comparisons++;
                if (t.charAt(s + j) != p.charAt(j)) break;  // mismatch
                j++;
            }
            if (j == m) hits.add(s);
        }
        return hits;
    }
    public static void main(String[] args) {
        System.out.println(search("abracadabra", "abra") + " " + comparisons);
        comparisons = 0;
        String t = "a".repeat(1000) + "b";                  // worst case
        String p = "a".repeat(10) + "b";
        System.out.println(search(t, p) + " " + comparisons + " comparisons");
        System.out.println("n * m = " + t.length() * p.length());
    }
}
```

Output codeoutput/NaiveSearch.txt

```
[0, 7] 16
[990] 10901 comparisons
n * m = 11011
```

On the worst case ($n = 1001$, $m = 11$) it makes 10 901 comparisons, close to $n \cdot m = 11\,011$.

Knuth–Morris–Pratt

KMP: never look back in the text

After q characters matched, we **know** $T[i-q..i-1] = P[0..q-1]$. On a mismatch, shift P so that its longest **border** (a proper prefix that is also a suffix) lines up with that text. The text index i never moves back.

Prefix function: $\pi[q]$ = length of the longest proper border of $P[0..q]$.

q	0	1	2	3	4	5	6
$P[q]$	a	b	a	b	a	c	a
$\pi[q]$	0	0	1	2	3	0	1

e.g. $\pi[4] = 3$: aba is both a prefix and a suffix of ababa.

Algorithm 2: PrefixFunction(P)

```
1 π[0] ← 0,    k ← 0
2 for q ← 1 to m − 1 do
3     while k > 0 and P[k] ≠ P[q] do
4         k ← π[k − 1]
5     if P[k] = P[q] then
6         k ← k + 1
7     π[q] ← k
8 return π
```

Amortized: k increases at most m times, so $O(m)$. Matching is the same loop over T: $O(n)$.

KMP in Java

codeoutput/KMP.java **Java**

```java
    static int[] prefix(String p) {        // pi[q]: longest
        border
        int[] pi = new int[p.length()];    //         of p[0..q]
        int k = 0;
        for (int q = 1; q < p.length(); q++) {
            while (k > 0 && p.charAt(k) != p.charAt(q)) {
                k = pi[k - 1];
            }
            if (p.charAt(k) == p.charAt(q)) k++;
            pi[q] = k;
        }
        return pi;
    }
```

codeoutput/KMP.java **Java**

```java
    static List<Integer> search(String t, String p) {
        int[] pi = prefix(p);
        List<Integer> hits = new ArrayList<>();
        int q = 0;                                  // characters matched
        for (int i = 0; i < t.length(); i++) {
            while (q > 0 && p.charAt(q) != t.charAt(i)) {
                comparisons++;
                q = pi[q - 1];                      // fall back, i stays
            }
            comparisons++;
            if (p.charAt(q) == t.charAt(i)) q++;
            if (q == p.length()) {                  // match ends at i
                hits.add(i - q + 1);
                q = pi[q - 1];
            }
        }
        return hits;
    }
```

KMP in Java

codeoutput/KMP.java — Java

```java
    static int[] prefix(String p) {        // pi[q]: longest
        border
        int[] pi = new int[p.length()];    //         of p[0..q]
        int k = 0;
        for (int q = 1; q < p.length(); q++) {
            while (k > 0 && p.charAt(k) != p.charAt(q)) {
                k = pi[k - 1];
            }
            if (p.charAt(k) == p.charAt(q)) k++;
            pi[q] = k;
        }
        return pi;
    }
```

Output — codeoutput/KMP.txt

```
[0, 0, 1, 2, 3, 0, 1]
[0, 7]
[990] 1991 comparisons
```

Same worst case: 1 991 comparisons instead of 10 901. Each text character is compared at most twice (once matched, once after a fall-back): at most $2n$, so $O(n + m)$.

codeoutput/KMP.java — Java

```java
    static List<Integer> search(String t, String p) {
        int[] pi = prefix(p);
        List<Integer> hits = new ArrayList<>();
        int q = 0;                                  // characters matched
        for (int i = 0; i < t.length(); i++) {
            while (q > 0 && p.charAt(q) != t.charAt(i)) {
                comparisons++;
                q = pi[q - 1];                      // fall back, i stays
            }
            comparisons++;
            if (p.charAt(q) == t.charAt(i)) q++;
            if (q == p.length()) {                  // match ends at i
                hits.add(i - q + 1);
                q = pi[q - 1];
            }
        }
        return hits;
    }
```

Rabin–Karp

Rabin–Karp: compare hashes, not strings

Treat a window of m characters as a number in base B, modulo a prime Q:

$$h(s) = \left(\sum_{j=0}^{m-1} T[s+j] \cdot B^{m-1-j} \right) \bmod Q$$

Sliding the window by one is $O(1)$, not $O(m)$:

$$h(s{+}1) = \left((h(s) - T[s] \cdot B^{m-1}) \cdot B + T[s{+}m] \right) \bmod Q$$

Only when $h(s) = h(P)$ compare the characters (hashes may collide).

Cost

Expected $O(n+m)$; worst case $O(nm)$ if many hashes collide.

Where it shines

Searching **many patterns** at once (put all pattern hashes in a set), and plagiarism detection (hash every window of k words).

`String.hashCode()` is the same polynomial with $B = 31$ and $Q = 2^{32}$ (overflow).

Rabin–Karp in Java

codeoutput/RabinKarp.java — Java

```java
public class RabinKarp {
    static final long B = 256, Q = 1_000_000_007L;      // base, prime modulus

    static int search(String t, String p) {             // first match or -1
        int n = t.length(), m = p.length();
        if (m > n) return -1;
        long hp = 0, ht = 0, pow = 1;                    // pow = B^(m-1) mod Q
        for (int i = 0; i < m; i++) {
            hp = (hp * B + p.charAt(i)) % Q;
            ht = (ht * B + t.charAt(i)) % Q;
            if (i > 0) pow = pow * B % Q;
        }
        for (int s = 0; ; s++) {
            if (hp == ht && t.regionMatches(s, p, 0, m)) return s;   // verify
            if (s + m >= n) return -1;
            ht = (ht - t.charAt(s) * pow % Q + Q) % Q;  // drop left char
            ht = (ht * B + t.charAt(s + m)) % Q;        // add right char
        }
    }

    public static void main(String[] args) {
        String t = "yeditepe university";
        System.out.println(search(t, "tepe"));
        System.out.println(search(t, "versi"));
        System.out.println(search(t, "java"));
    }
}
```

`regionMatches` verifies a hash hit, so a collision can never give a wrong answer.

Rabin–Karp in Java

codeoutput/RabinKarp.java — Java

```java
public class RabinKarp {
    static final long B = 256, Q = 1_000_000_007L;      // base, prime modulus

    static int search(String t, String p) {             // first match or -1
        int n = t.length(), m = p.length();
        if (m > n) return -1;
        long hp = 0, ht = 0, pow = 1;                    // pow = B^(m-1) mod Q
        for (int i = 0; i < m; i++) {
            hp = (hp * B + p.charAt(i)) % Q;
            ht = (ht * B + t.charAt(i)) % Q;
            if (i > 0) pow = pow * B % Q;
        }
        for (int s = 0; ; s++) {
            if (hp == ht && t.regionMatches(s, p, 0, m)) return s;   // verify
            if (s + m >= n) return -1;
            ht = (ht - t.charAt(s) * pow % Q + Q) % Q;  // drop left char
            ht = (ht * B + t.charAt(s + m)) % Q;        // add right char
        }
    }

    public static void main(String[] args) {
        String t = "yeditepe university";
        System.out.println(search(t, "tepe"));
        System.out.println(search(t, "versi"));
        System.out.println(search(t, "java"));
    }
}
```

Output codeoutput/RabinKarp.txt

```
4
12
-1
```

`regionMatches` verifies a hash hit, so a collision can never give a wrong answer.

Edit Distance

Edit distance (Levenshtein)

The fewest single-character **insertions**, **deletions** and **replacements** that turn a into b. `kitten` → `sitting` needs 3: **k**itten → **s**itten → sitt**i**n → sitting**.**

$d[i][j]$ = distance between the first i characters of a and the first j of b:

$$d[i][j] = \min \begin{cases} d[i-1][j-1] + [a_i \neq b_j] & \text{replace or keep} \\ d[i-1][j] + 1 & \text{delete } a_i \\ d[i][j-1] + 1 & \text{insert } b_j \end{cases}$$

with $d[i][0] = i$ and $d[0][j] = j$.

- **Dynamic programming**: fill the table row by row
- Time and space $O(|a| \cdot |b|)$; space $O(|b|)$ if only the distance is needed (keep two rows)
- Uses: spelling suggestions (“did you mean”), DNA alignment, `diff`, fuzzy search

Edit distance in Java

codeoutput/EditDistance.java — Java

```java
public class EditDistance {
    static int[][] table(String a, String b) {
        int[][] d = new int[a.length() + 1][b.length() + 1];
        for (int i = 0; i <= a.length(); i++) d[i][0] = i;     // delete all
        for (int j = 0; j <= b.length(); j++) d[0][j] = j;     // insert all
        for (int i = 1; i <= a.length(); i++) {
            for (int j = 1; j <= b.length(); j++) {
                int sub = a.charAt(i - 1) == b.charAt(j - 1) ? 0 : 1;
                d[i][j] = Math.min(d[i - 1][j - 1] + sub,          // replace/keep
                          Math.min(d[i - 1][j] + 1,                // delete
                                   d[i][j - 1] + 1));              // insert
            }
        }
        return d;
    }
    public static void main(String[] args) {
        String a = "kitten", b = "sitting";
        int[][] d = table(a, b);
        System.out.println("       " + String.join("  ", b.split("")));
        for (int i = 0; i <= a.length(); i++) {
            System.out.print(i == 0 ? "  " : " " + a.charAt(i - 1));
            for (int j = 0; j <= b.length(); j++) System.out.printf("%3d", d[i][j]);
            System.out.println();
        }
        System.out.println("distance = " + d[a.length()][b.length()]);
    }
}
```

The bottom-right cell is the answer; following the minima back gives the edits.

Edit distance in Java

codeoutput/EditDistance.java — Java

```java
public class EditDistance {
    static int[][] table(String a, String b) {
        int[][] d = new int[a.length() + 1][b.length() + 1];
        for (int i = 0; i <= a.length(); i++) d[i][0] = i;      // delete all
        for (int j = 0; j <= b.length(); j++) d[0][j] = j;      // insert all
        for (int i = 1; i <= a.length(); i++) {
            for (int j = 1; j <= b.length(); j++) {
                int sub = a.charAt(i - 1) == b.charAt(j - 1) ? 0 : 1;
                d[i][j] = Math.min(d[i - 1][j - 1] + sub,           // replace/keep
                          Math.min(d[i - 1][j] + 1,                 // delete
                                   d[i][j - 1] + 1));               // insert
            }
        }
        return d;
    }
    public static void main(String[] args) {
        String a = "kitten", b = "sitting";
        int[][] d = table(a, b);
        System.out.println("       " + String.join("  ", b.split("")));
        for (int i = 0; i <= a.length(); i++) {
            System.out.print(i == 0 ? "  " : " " + a.charAt(i - 1));
            for (int j = 0; j <= b.length(); j++) System.out.printf("%3d", d[i][j]);
            System.out.println();
        }
        System.out.println("distance = " + d[a.length()][b.length()]);
    }
}
```

Output
codeoutput/EditDistance.txt

```
      s  i  t  t  i  n  g
   0  1  2  3  4  5  6  7
k  1  1  2  3  4  5  6  7
i  2  2  1  2  3  4  5  6
t  3  3  2  1  2  3  4  5
t  4  4  3  2  1  2  3  4
e  5  5  4  3  2  2  3  4
n  6  6  5  4  3  3  2  3
distance = 3
```

The bottom-right cell is the answer; following the minima back gives the edits.

Regular Expressions

Regular expressions in one slide

Pattern	Matches
`.`	any character
`[abc]`, `[^abc]`	one of / none of
`[a-z]`	a range
`\d`, `\s`, `\w`	digit, space, word char (ASCII)
`\p{L}`	a letter in *any* language
`X*`, `X+`, `X?`	0+, 1+, 0 or 1 times
`X{2,4}`	2 to 4 times
`^`, `$`	start, end of input
`(X)`, `(?<name>X)`	group, named group
`X|Y`	`X` or `Y`

In Java

- In a string literal every `\` is doubled: the regex `\d+` is written `"\\d+"`
- `s.matches(re)`: the **whole** string must match
- `Matcher.find()`: the next match **anywhere**
- `split`, `replaceAll`, `replaceFirst` take regexes
- Compile once, reuse: `Pattern.compile(re)`
- In a replacement, `$1` refers to group 1

Regular expressions in Java

codeoutput/RegexDemo.java — Java

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class RegexDemo {
    public static void main(String[] args) {
        String text = "Exams: 2026-01-12 (CSE 101), 2026-01-15 (CSE 211).";
        Pattern date = Pattern.compile("(\\d{4})-(\\d{2})-(\\d{2})");
        Matcher m = date.matcher(text);
        while (m.find()) {
            System.out.println(m.group() + ": day " + m.group(3));
        }
        Pattern course = Pattern.compile("(?<dept>[A-Z]{3}) (?<num>\\d{3})");
        Matcher c = course.matcher(text);
        while (c.find()) {
            System.out.println(c.group("dept") + "/" + c.group("num"));
        }
        System.out.println(date.matcher(text).replaceAll("$3.$2.$1"));
        String mail = "[\\w.]+@[\\w.]+\\.[a-z]{2,}";
        System.out.println("ali.veli@example.edu.tr".matches(mail));
        System.out.println("Çağrı".matches("\\w+"));       // ASCII only
        System.out.println("Çağrı".matches("\\p{L}+"));    // any letter
    }
}
```

`\w` is ASCII-only in Java, so `"Çağrı".matches("\\w+")` is `false`; `\p{L}` accepts letters of every language.

Regular expressions in Java

codeoutput/RegexDemo.java — Java

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class RegexDemo {
    public static void main(String[] args) {
        String text = "Exams: 2026-01-12 (CSE 101), 2026-01-15 (CSE 211).";
        Pattern date = Pattern.compile("(\\d{4})-(\\d{2})-(\\d{2})");
        Matcher m = date.matcher(text);
        while (m.find()) {
            System.out.println(m.group() + ": day " + m.group(3));
        }
        Pattern course = Pattern.compile("(?<dept>[A-Z]{3}) (?<num>\\d{3})");
        Matcher c = course.matcher(text);
        while (c.find()) {
            System.out.println(c.group("dept") + "/" + c.group("num"));
        }
        System.out.println(date.matcher(text).replaceAll("$3.$2.$1"));
        String mail = "[\\w.]+@[\\w.]+\\.[a-z]{2,}";
        System.out.println("ali.veli@example.edu.tr".matches(mail));
        System.out.println("Çağrı".matches("\\w+"));       // ASCII only
        System.out.println("Çağrı".matches("\\p{L}+"));    // any letter
    }
}
```

Output codeoutput/RegexDemo.txt

```
2026-01-12: day 12
2026-01-15: day 15
CSE/101
CSE/211
Exams: 12.01.2026 (CSE 101), 15.01.2026 (CSE 211).
true
false
true
```

`\w` is ASCII-only in Java, so `"Çağrı".matches("\\w+")` is `false`; `\p{L}` accepts letters of every language.

Unicode and Encodings

chars, code points and grapheme clusters

Three ways to count "characters":

- `char`: a 16-bit UTF-16 code unit; `length()` counts these
- **code point**: a Unicode number `U+0000..U+10FFFF`; above `U+FFFF` it takes two `chars` (a **surrogate pair**)
- **grapheme cluster**: what a reader sees as one letter; may be several code points (e + combining accent, flags, family emoji)

Unit	Java API
char	`length()`, `charAt`
code point	`codePoints()`, `codePointAt`
grapheme	`BreakIterator`, regex `\X`

Case and locale

Case mapping can change the length: `"İ".toLowerCase(Locale.ROOT)` is `i` + a combining dot (2 chars).

Unicode in Java

codeoutput/UnicodeDemo.java — Java

```java
import java.text.Normalizer;
import java.util.Locale;

public class UnicodeDemo {
    static String hex(String s) {                          // chars as hex codes
        StringBuilder sb = new StringBuilder();
        s.chars().forEach(c -> sb.append(String.format("\\u%04X", c)));
        return sb.toString();
    }
    public static void main(String[] args) {
        String smile = "Hi \uD83D\uDE00";                  // "Hi " + U+1F600
        System.out.println(smile.length() + " chars, "
                + smile.codePointCount(0, smile.length()) + " code points");
        smile.codePoints().forEach(cp -> System.out.printf("U+%04X ", cp));
        System.out.println();
        String rev = new StringBuilder(smile).reverse().toString();
        System.out.println("pair kept by reverse(): " + (rev.codePointAt(0) == 0x1F600));
        String dotI = "\u0130";                                // capital dotted I
        System.out.println("ROOT lower: " + hex(dotI.toLowerCase(Locale.ROOT)));
        System.out.println("tr   lower: " + hex(dotI.toLowerCase(Locale.forLanguageTag("tr"))));
        String e1 = "\u00E9", e2 = "e\u0301";              // e-acute, e + accent
        String n2 = Normalizer.normalize(e2, Normalizer.Form.NFC);
        System.out.println(hex(e1) + " vs " + hex(e2) + ": " + e1.equals(e2));
        System.out.println("after NFC: " + e1.equals(n2));
    }
}
```

`reverse()` keeps surrogate pairs; equal-looking strings may differ until **normalized** (NFC).

Unicode in Java

codeoutput/UnicodeDemo.java — Java

```java
import java.text.Normalizer;
import java.util.Locale;

public class UnicodeDemo {
    static String hex(String s) {                          // chars as hex codes
        StringBuilder sb = new StringBuilder();
        s.chars().forEach(c -> sb.append(String.format("\\u%04X", c)));
        return sb.toString();
    }
    public static void main(String[] args) {
        String smile = "Hi \uD83D\uDE00";                  // "Hi " + U+1F600
        System.out.println(smile.length() + " chars, "
                + smile.codePointCount(0, smile.length()) + " code points");
        smile.codePoints().forEach(cp -> System.out.printf("U+%04X ", cp));
        System.out.println();
        String rev = new StringBuilder(smile).reverse().toString();
        System.out.println("pair kept by reverse(): " + (rev.codePointAt(0) == 0x1F600));
        String dotI = "\u0130";                                // capital dotted I
        System.out.println("ROOT lower: " + hex(dotI.toLowerCase(Locale.ROOT)));
        System.out.println("tr   lower: " + hex(dotI.toLowerCase(Locale.forLanguageTag("tr"))));
        String e1 = "\u00E9", e2 = "e\u0301";              // e-acute, e + accent
        String n2 = Normalizer.normalize(e2, Normalizer.Form.NFC);
        System.out.println(hex(e1) + " vs " + hex(e2) + ": " + e1.equals(e2));
        System.out.println("after NFC: " + e1.equals(n2));
    }
}
```

Output codeoutput/UnicodeDemo.txt

```
5 chars, 4 code points
U+0048 U+0069 U+0020 U+1F600
pair kept by reverse(): true
ROOT lower: \u0069\u0307
tr   lower: \u0069
\u00E9 vs \u0065\u0301: false
after NFC: true
```

`reverse()` keeps surrogate pairs; equal-looking strings may differ until **normalized** (NFC).

Bytes and encodings

codeoutput/Encoding.java — Java

```java
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;

public class Encoding {
    public static void main(String[] args) {
        String s = "Äğaç";
        for (String cs : new String[] {"UTF-8", "ISO-8859-9", "UTF-16BE"}) {
            byte[] b = s.getBytes(Charset.forName(cs));
            StringBuilder hex = new StringBuilder();
            for (byte x : b) hex.append(String.format("%02X ", x));
            System.out.printf("%-10s %d bytes: %s%n", cs, b.length, hex);
        }
        byte[] utf8 = s.getBytes(StandardCharsets.UTF_8);
        System.out.println(new String(utf8, Charset.forName("windows-1252")));  // wrong
        System.out.println(new String(utf8, StandardCharsets.UTF_8));            // right
        System.out.println("default charset: " + Charset.defaultCharset());
    }
}
```

Decoding UTF-8 bytes as Windows-1252 gives `AÄŸaÃ§`. Since Java 18 the default charset is UTF-8 on every system.

Bytes and encodings

codeoutput/Encoding.java — Java

```java
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;

public class Encoding {
    public static void main(String[] args) {
        String s = "Ağaç";
        for (String cs : new String[] {"UTF-8", "ISO-8859-9", "UTF-16BE"}) {
            byte[] b = s.getBytes(Charset.forName(cs));
            StringBuilder hex = new StringBuilder();
            for (byte x : b) hex.append(String.format("%02X ", x));
            System.out.printf("%-10s %d bytes: %s%n", cs, b.length, hex);
        }
        byte[] utf8 = s.getBytes(StandardCharsets.UTF_8);
        System.out.println(new String(utf8, Charset.forName("windows-1252")));  // wrong
        System.out.println(new String(utf8, StandardCharsets.UTF_8));            // right
        System.out.println("default charset: " + Charset.defaultCharset());
    }
}
```

Output — codeoutput/Encoding.txt

```
UTF-8      6 bytes: 41 C4 9F 61 C3 A7
ISO-8859-9 4 bytes: 41 F0 61 E7
UTF-16BE   8 bytes: 00 41 01 1F 00 61 00 E7
AÄŸaÃ§
Ağaç
default charset: UTF-8
```

Decoding UTF-8 bytes as Windows-1252 gives `AÄŸaÃ§`. Since Java 18 the default charset is UTF-8 on every system.

Encodings on Windows and macOS

Windows

- Old Java (before 18): the default charset was Windows-1254 (Turkish), files written there were not UTF-8
- Console: run `chcp 65001`; Java 19+ also reads it from `stdout.encoding`
- Compile UTF-8 sources with `javac -encoding UTF-8`
- Notepad saves UTF-8 by default since Windows 10 (1903)

macOS

- Terminal and the file system use UTF-8
- Older HFS+ volumes store file names decomposed (NFD): ç may arrive as c + cedilla; **normalize** names before comparing
- TextEdit: choose UTF-8 in the save dialog for plain text

Rule for portable programs

Always pass a charset: `Files.readString(path, StandardCharsets.UTF_8)`, `new InputStreamReader(in, UTF_8)`. Never rely on the platform default.

Summary

String matching algorithms at a glance

Algorithm	Preprocess	Match	When to use
Naive	none	$O(nm)$ worst	short patterns, typical text (`indexOf`)
KMP	$O(m)$	$O(n)$	guaranteed linear time, streaming text
Rabin–Karp	$O(m)$	$O(n)$ expected	many patterns, plagiarism (window hashes)
Boyer–Moore	$O(m + \lvert\Sigma\rvert)$	sublinear typical	long patterns, large alphabets (`grep`)
Aho–Corasick	$O(\sum m_i)$	$O(n + \text{matches})$	a dictionary of patterns at once
Suffix array	$O(n \log n)$	$O(m \log n)$	many queries on one fixed text

Edit distance: $O(\lvert a\rvert \cdot \lvert b\rvert)$ dynamic programming. Regex: convenient, but its cost depends on the pattern (see the senior slides on catastrophic backtracking).

Exercises

1. Compute π for `aabaaab` by hand, then check with `KMP.prefix`.
2. Extend `NaiveSearch` and `KMP` to count comparisons on random texts over $\{a, b\}$; plot both against n.
3. Use Rabin–Karp with k-word windows to find the longest common passage of two text files.
4. Print the actual edits (replace / insert / delete) from the edit-distance table.
5. Write a regex for Turkish mobile numbers (`05xx xxx xx xx`, spaces optional) and test it.
6. Reverse a string by **code points**, and show that a `char` loop breaks an emoji.
7. Read a UTF-8 file with Turkish text on Windows and macOS; compare with and without an explicit charset.

Questions?